

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Факультет безопасности информационных технологий

Образовательная программа: Технологии защиты информации

Направление подготовки (специальность): 10.03.01 Информационная безопасность

О Т Ч Е Т
об учебной практике

Тема задания: Анализ логических ошибок контроля доступа и разработка сценариев тестирования в условиях соревнований Attack-Defense CTF

Обучающийся:

студентка группы N3446 Чан Бао Линь

Согласовано:

Руководитель практики в ИТМО: Меншиков Александр Алексеевич, доцент ФБИТ, к.т.н., доцент, декан ФБИТ, директор учебно-практического центра - Киберполигон

Ответственный за практику от университета: Еритенко Николай Алексеевич, инженер ФБИТ

Практика пройдена с оценкой _____

Дата 13.02.2026

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Содержание.....	2
Введение.....	3
1 Общая часть.....	5
1.1 Формат Attack-Defense CTF и критерии работоспособности.....	5
1.2 Контроль доступа в веб-сервисах: аутентификация и авторизация.....	6
1.3 Типовые логические ошибки контроля доступа.....	8
1.4 Подход к тестированию контроля доступа в условиях Attack-Defense.....	11
1.5 Инструменты и артефакты проверки.....	13
2 Практическая часть.....	15
2.1 Условия соревнования и стенд эксперимента.....	15
2.2 Краткое описание сервиса и модели данных.....	16
2.3 Поверхность атаки и точки контроля доступа (Otkritki/Zapiski).....	18
2.4 Анализ логических ошибок контроля доступа.....	20
2.5 Разработка сценариев тестирования и тест-матрицы.....	21
2.6 Рекомендации и валидация.....	27
Заключение.....	29
Список использованных источников.....	30

ВВЕДЕНИЕ

Формат Attack-Defense CTF представляет собой практико-ориентированную модель соревнований, в которой команды одновременно обеспечивают работоспособность своих сервисов и противодействуют атакам соперников. В таких условиях ключевую роль играет сохранность приватных данных пользователей (в игровых сервисах - флагов), а также стабильность функционала, проверяемого чекерами. На практике одной из наиболее частых причин утечек в веб-сервисах становятся логические ошибки контроля доступа: некорректные проверки владельца объекта, несогласованность правил авторизации в разных эндпоинтах, ошибки обработки сессий и параметров запросов. Подобные дефекты приводят к прямому компрометированию приватных данных и потерям очков, а попытки «жёстко» закрыть уязвимости без учёта сценариев чекера могут ухудшить доступность и корректность сервиса (снижение SLA).

Актуальность исследования обусловлена тем, что в условиях A/D-соревнований ошибки контроля доступа проявляются особенно наглядно: сервисы активно используются автоматизированными проверками и одновременно подвергаются интенсивному внешнему воздействию. Следовательно, требуется подход, который позволяет системно выявлять логические дефекты доступа и разрабатывать тест-сценарии, обеспечивающие воспроизводимость результатов и последующую валидацию исправлений без деградации функциональности.

Цель работы - выполнить анализ логических ошибок контроля доступа в веб-сервисе в условиях Attack-Defense CTF и разработать набор тест-сценариев для их выявления и проверки исправлений с учётом требований к работоспособности сервиса.

Объект исследования - механизмы контроля доступа в веб-сервисах, функционирующих в формате Attack-Defense CTF.

Предмет исследования - логические ошибки авторизации/проверок прав доступа и методика построения тест-сценариев для их обнаружения и валидации.

Практическая база - стенд соревнований (vulnbox/контейнерная инфраструктура) и сервис(ы) типа Otkritki/Zapiski.

Для достижения цели решаются следующие задачи (в сокращённом виде):

- описать специфику A/D-формата и связь утечек приватных данных со статусами работоспособности и SLA;
- выделить типовые классы логических ошибок контроля доступа (в т.ч. ошибки объектной авторизации/IDOR, несогласованные проверки, уязвимости сессий);

- проанализировать поверхность атаки сервиса: эндпоинты, сессионную модель, правила доступа к объектам данных;
- разработать тест-сценарии (positive/negative/edge) и критерии pass/fail для проверки контроля доступа;
- выполнить тестирование на стенде, зафиксировать результаты и подготовить рекомендации и повторные проверки (валидацию).

Методы и средства исследования включают анализ исходного кода и конфигурации, функциональное тестирование API, анализ журналов и диагностических данных, а также наблюдение сетевого взаимодействия (при наличии инструментов мониторинга трафика). Результаты оформляются в виде тест-матрицы, протоколов проверок и набора рекомендаций по усилению контроля доступа без нарушения требований чекера.

Практическая значимость работы заключается в том, что предложенный набор тест-сценариев и подход к их применению может использоваться как для ускорения аудита сервисов в соревнованиях Attack-Defense CTF, так и как учебный шаблон для анализа ошибок авторизации в прикладных веб-системах.

Отчёт структурирован в два крупных раздела:

1. обзорный - формат A/D, контроль доступа и типовые логические уязвимости;
2. практический - анализ сервиса на стенде, разработка и применение тест-сценариев, результаты, рекомендации и валидация.

1 ОБЩАЯ ЧАСТЬ

1.1 Формат Attack-Defense CTF и критерии работоспособности

Attack-Defense CTF - это соревнование в реальном времени, где каждая команда одновременно:

- защищает свои сервисы (чтобы не допустить утечек приватных данных),
- атакует сервисы других команд (чтобы извлекать приватные данные и получать очки).

Раунды: Игра разбита на раунды - фиксированные интервалы времени. В каждом раунде для каждого сервиса команды выполняется автоматическая проверка.

Чекер - программа организаторов, которая эмулирует поведение обычного пользователя и проверяет, что сервис:

- доступен по сети,
- корректно выполняет ключевые функции (регистрация/логин/создание/чтение и т.п.),
- сохраняет и возвращает ранее размещённые приватные данные.

Чекер также кладёт новые приватные данные в сервис.

Флаг - это строка приватных данных, которую чекер сохраняет в сервисе. Обычно флаги имеют узнаваемый формат, например по регулярному выражению: `[A-Z0-9]{31}=`

По смыслу флаг должен быть доступен только “своему” пользователю (владельцу) и недоступен другим.

Статусы сервиса (OK / MUMBLE / CORRUPT / DOWN)

По итогам раунда сервис получает один из статусов:

- OK - сервис работает корректно, все необходимые функции проходят.
- MUMBLE - сервис доступен, но часть функционала работает неверно (например, сломан нужный сценарий чекера).
- CORRUPT - данные пользователя “повреждены”: невозможно получить ранее сохранённые приватные данные (например, флаги пропали или читаются неправильно).
- DOWN - сервис недоступен по сети.

SLA (Service Level Agreement)

SLA - ключевая метрика стабильности сервиса: доля раундов, в которых сервис имел статус OK.

Высокий SLA означает, что сервис стабильно проходит проверки чекера, а низкий SLA сильно снижает итоговый результат команды.

Связь с очками

В Attack-Defense обычно начисляются очки за украденные флаги (FP, flag points), а итоговая ценность часто зависит от произведения $SLA \times FP$. Поэтому важно:

- не допускать утечек флагов (иначе теряются FP),
- не ломать функциональность (иначе падает SLA из-за MUMBLE/CORRUPT/DOWN).

Round: 185					
UP		CORRUPT	MUMBLE	DOWN	CHECK FAILED
#	team	score	Zapiski		otkritki
1	WELCOME 172.25.3.2	7866.63	SLA: 97.30% FP: 5612.93 +346/-52	SLA: 96.22% FP: 2500.00 +0/-0	
2	ООО ПЕНЕТРЕЙТОР\$\$ 172.25.1.2	7782.44	SLA: 100.00% FP: 5186.11 +85/-72	SLA: 100.00% FP: 2596.32 +10/-8	
3	SSH 172.25.5.2	5075.95	SLA: 99.46% FP: 960.94 +0/-84	SLA: 99.46% FP: 4142.59 +31/-0	
4	Ливерпуль 172.25.4.2	3354.33	SLA: 100.00% FP: 1143.92 +0/-73	SLA: 98.38% FP: 2246.84 +0/-5	
5	Кабанчики 172.25.2.2	2816.55	SLA: 99.46% FP: 1143.73 +0/-72	SLA: 100.00% FP: 1679.00 +0/-25	
6	NPC 172.25.6.2	2147.55	SLA: 98.92% FP: 952.36 +0/-78	SLA: 98.38% FP: 1835.24 +0/-13	

Рисунок 1 - Итоговая рейтинговая таблица второго соревнования

1.2 Контроль доступа в веб-сервисах: аутентификация и авторизация

Контроль доступа в веб-сервисах - это набор механизмов, который обеспечивает, что пользователь:

1. является тем, за кого себя выдаёт,
2. может выполнять только разрешённые действия,
3. получает доступ только к тем данным, на которые имеет право.

Аутентификация (Authentication, AuthN)

Аутентификация отвечает на вопрос: “Кто ты?”

Это проверка личности пользователя (или сервиса/клиента) перед выдачей доступа.

Типовые варианты:

- логин/пароль (с хранением паролей в виде хэша),
- сессия (cookie) после успешного входа,

- токены (например, JWT), API-ключи,
- MFA (реже в CTF, но важно в реальных системах).

Результат AuthN: сервер создаёт сессию или выдаёт токен, по которому далее “узнаёт” пользователя.

Авторизация (Authorization, AuthZ)

Авторизация отвечает на вопрос: “Что тебе можно?”

Это проверка прав уже аутентифицированного пользователя на конкретное действие или объект данных:

- чтение/изменение/удаление ресурса,
- выполнение операции (создать карточку, открыть запись и т.д.).

Авторизация обычно строится на трёх подходах:

1. Ownership (владение объектом)

Пользователь может работать только со “своими” объектами: my cards / my notes.

Это самый частый механизм в A/D: именно он защищает флаги от чужих пользователей.

2. Roles (роли, RBAC)

Права зависят от роли: user / admin / moderator.

В сервисах A/D часто встречаются “особые роли” или ограничения по признаку (например, доступ только определённой группе).

3. Attributes (ABAC)

Решение зависит от атрибутов пользователя/объекта/контекста (время, статус, тип операции и т.п.).

Это полезно, когда простых ролей недостаточно.

Принцип «проверки на сервере»

Главное правило: все проверки доступа должны выполняться на сервере для каждого запроса.

Ограничения на фронтенде (скрытая кнопка, запрет в UI) не являются защитой, потому что злоумышленник может отправить запрос напрямую (curl/Burp/скрипт).

То есть безопасность обеспечивается не тем, что “пользователь не видит кнопку”, а тем, что сервер при запросе:

- определяет пользователя (AuthN),
- проверяет права (AuthZ),
- и только потом отдаёт данные/выполняет действие.

Почему это важно именно в Attack-Defense

В A/D флаги хранятся внутри обычной логики сервиса. Если авторизация реализована неверно (например, объект можно запросить по id без строгой проверки владельца), это приводит к утечке флагов и потере очков. При этом слишком “жёсткие” или ошибочные проверки могут сломать сценарии чекера и вызвать MUMBLE/CORRUPT, снижая SLA. Поэтому контроль доступа в A/D - это баланс: не утекать и не ломать функциональность.

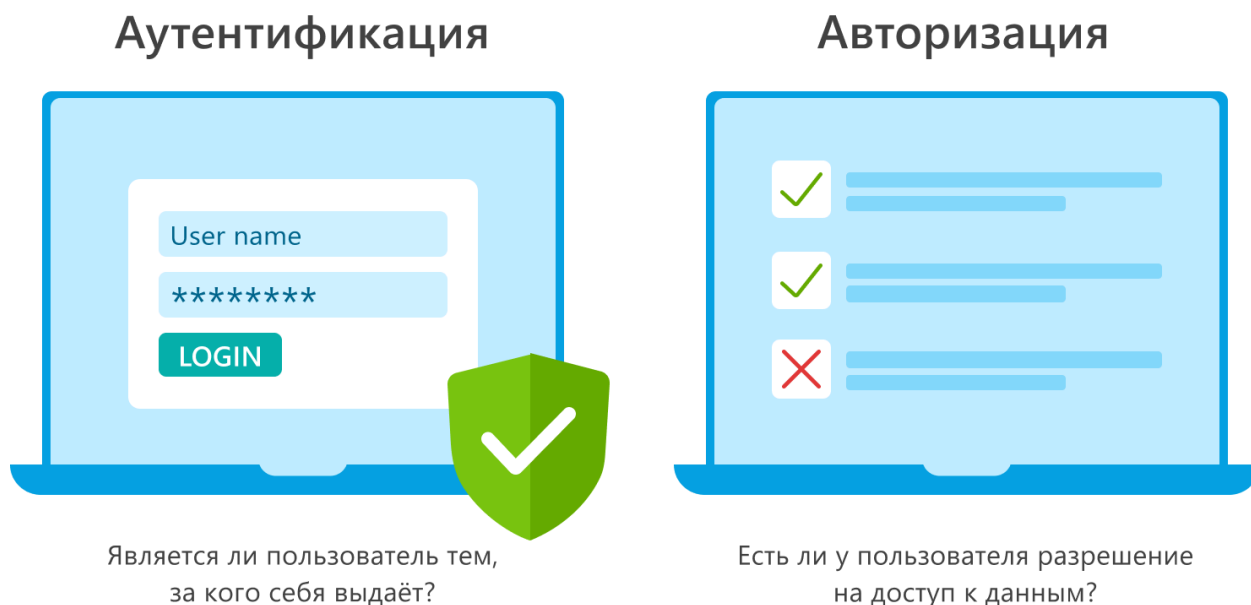


Рисунок 2 - Аутентификация и Авторизация

1.3 Типовые логические ошибки контроля доступа

Логические ошибки контроля доступа возникают не из-за “технического взлома”, а из-за неправильной бизнес-логики проверок прав. В веб-сервисах (и особенно в Attack-Defense CTF) они часто приводят к прямой утечке приватных данных (флагов) при сохранении видимой работоспособности сервиса.

1) IDOR / BOLA (Broken Object Level Authorization)

Суть: доступ к объектам по идентификатору (id, card_id, note_id) без строгой проверки прав.

Пример: пользователь запрашивает /api/card?id=123 и получает чужой объект, потому что сервер не проверил владельца/получателя.

Последствие: массовое чтение/кража флагов; часто даёт статус CORRUPT у жертвы.

Broken Authorization

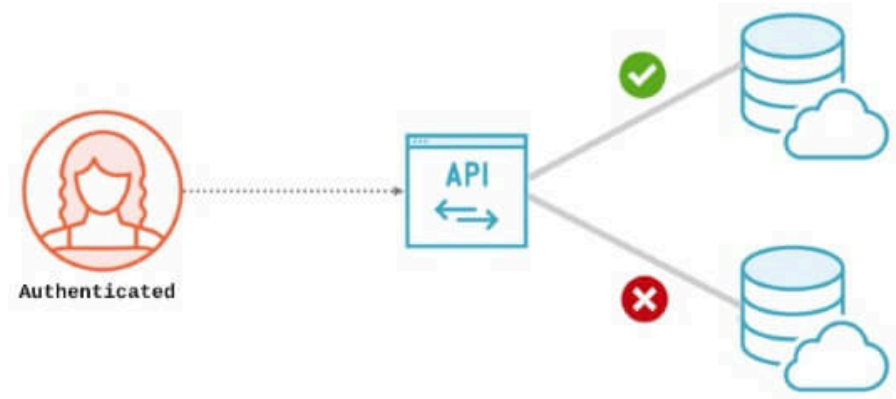


Рисунок 3 - (Broken Object Level Authorization)

2) Несогласованные проверки между эндпоинтами

Суть: в одном месте проверка прав есть, в другом - слабее или отсутствует.

Типичный случай:

- список “моих объектов” защищён (/api/cards отдаёт только своё),
- а “детальный просмотр” по id защищён плохо (/api/card?id=...) -> обход через прямой запрос.

Последствие: утечка приватных данных, хотя “обычный пользователь” в интерфейсе её не видит.

3) Ошибки проверок по строкам (substring-checks)

Суть: права проверяются “по подстроке”, частичному совпадению или небезопасному сравнению строк.

Пример: условие вида “если card.To содержит username”, а не строгое совпадение адресата/владельца.

Последствие: обход авторизации через похожие имена, форматирование, списки получателей, “коллизии” строк.

4) Ошибки логики условий (logic bugs)

Суть: неправильные ветвления if/else, ошибки AND/OR, неверные отрицания, пропущенные условия.

Типовые формы:

- проверяют “не то поле” (например, from вместо to),

- инверсия логики (“если не владелец - разрешить”),
- проверка выполняется только при определённых параметрах (например, если latest задан - проверка одна, если нет - другая).

Последствие: выбор “правильного” входа/параметра даёт обход контроля доступа.

5) Проблемы с сессиями (session issues)

Суть: ошибки в создании/хранении/проверке сессий приводят к подмене личности или к нестабильной авторизации.

Примеры проблем:

- отсутствие проверки пароля при логине (логин “по имени”),
- предсказуемые/общие ключи подписи cookie,
- некорректная обработка “новой” или пустой сессии,
- небезопасные параметры cookie (нет HttpOnly, SameSite, Secure - по условиям).

Последствие: захват сессии, обход AuthN, либо падения/нестабильность (риски DOWN/MUMBLE).

6) CSRF при cookie-сессиях (если есть state-changing запросы)

Суть: если сервис использует cookie-сессию и принимает POST/PUT/DELETE без CSRF-защиты, злоумышленник может заставить браузер жертвы выполнить действие “от её имени”.

Важно: CSRF не всегда напрямую ворует данные, но может:

- менять настройки/права,
- создавать/удалять объекты,
- приводить к CORRUPT (флаги “исчезли” или стали недоступны владельцу).

Почему эти ошибки особенно опасны в A/D

- Они часто дают тихую утечку: сервис продолжает работать (SLA высокий), но флаги читаются чужими.
- Попытка “закрыть всё подряд” без понимания сценариев чекера может сломать функционал -> MUMBLE/CORRUPT и падение SLA.

Поэтому в отчёте важно связывать каждую ошибку с двумя последствиями: утечка флагов и влияние на SLA/статусы.

Attack/defense CTF Network example

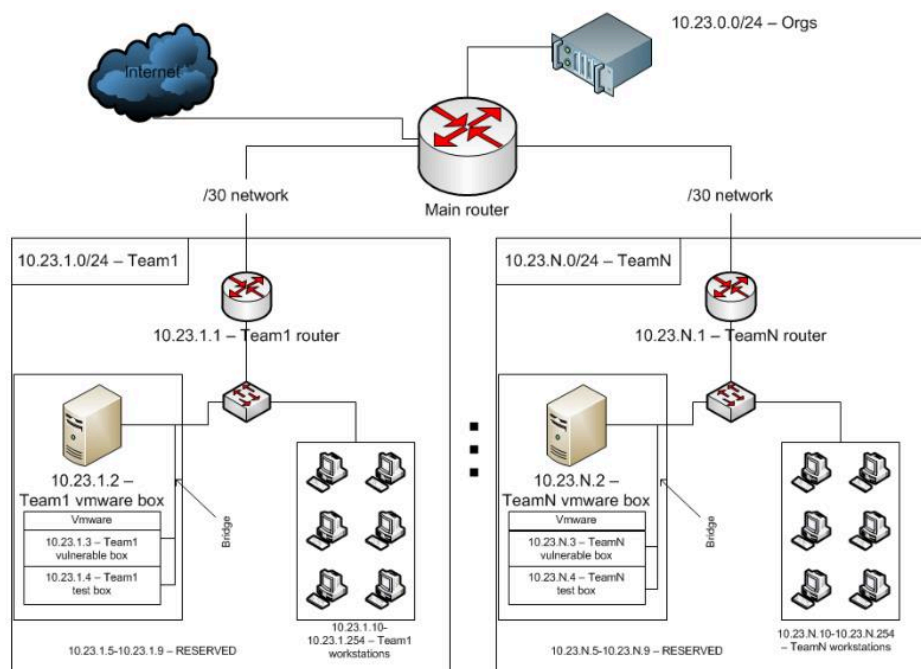


Рисунок 4 — Сетевая схема Attack-Defense CTF

Тестирование контроля доступа в A/D отличается тем, что нужно одновременно:

- не допустить утечек приватных данных (флагов),
- не сломать функциональность чекера, иначе сервис получит MUMBLE/CORRUPT и упадёт SLA.

Критерии pass/fail

Для каждого тест-кейса заранее задаются измеримые критерии:

- PASS (успех), если:
 - “свой” пользователь получает доступ к “своим” данным и операция выполняется корректно;
 - “чужой” пользователь не получает доступ к чужим данным (нет содержимого, нет метаданных, нет побочных утечек);
 - сервис отвечает стабильно (нет 5xx, паник, зависаний) и поведение повторяемо.
- FAIL (провал), если:
 - чужой пользователь может прочитать/изменить/удалить объект, содержащий флаг;

- можно обойти проверку прав через параметры (id, latest, фильтры), разные эндпоинты или методы;
- запросы вызывают нестабильность (ошибки 5xx/DoS), что может привести к DOWN/MUMBLE.

Виды тестов (coverage)

1. Positive tests (позитивные) - проверяют легитимные сценарии чекера:
регистрация -> логин -> создание данных -> чтение своих данных -> повторное чтение ранее созданного.
2. Negative tests (негативные) - попытки нарушить контроль доступа:
доступ без сессии, доступ “чужим” пользователем, запрос чужого объекта по id, обход через альтернативный эндпоинт.
3. Edge tests (краевые) - проверяют устойчивость логики и обработку нестандартных входов:
id=0, id=-1, очень большой id, строковые значения вместо чисел, пустые параметры, дубли параметров, неожиданные комбинации (например, latest + id).

Приоритизация сценариев (что тестировать первым)

В A/D приоритет ставится по риску утечки флагов и влиянию на SLA:

1. Object-level доступ к данным с флагами
(чтение карточки/записки по id, списки “моих” объектов, фильтры “latest”, поиск и т.п.)
2. Проверки прав на изменение состояния
(создание/редактирование/удаление): если злоумышленник может удалить/переместить данные, это часто приводит к CORRUPT.
3. Сессии и авторизация на всех маршрутах
(логин, cookie, logout, повторное использование cookie, поведение при отсутствии cookie).

Главное правило A/D: «не ломать SLA»

Любое “усиление защиты” должно проходить минимальную регрессию:

- ключевые positive-сценарии чекера должны стабильно работать (иначе MUMBLE),
- ранее положенные данные должны извлекаться владельцем (иначе CORRUPT),
- ответы должны быть предсказуемыми и без 5xx (иначе DOWN/MUMBLE).

Артефакты, которые фиксируются, чтобы результаты были воспроизводимыми, для каждого теста фиксируется:

- запрос (URL/метод/параметры/заголовки/cookie),

- ответ (код, тело, важные заголовки),
- подтверждение из логов (по возможности) и итог pass/fail.

Такой подход позволяет быстро выявлять логические ошибки контроля доступа, а также безопасно проверять исправления без ухудшения работоспособности сервиса в условиях Attack-Defense CTF.

1.5 Инструменты и артефакты проверки

Для анализа логических ошибок контроля доступа и воспроизводимого тестирования в условиях Attack-Defense CTF используются как средства взаимодействия с сервисом, так и инструменты наблюдения (диагностики).

Инструменты (tools)

- curl - быстрые ручные проверки API/HTTP: регистрация/логин, работа с cookie, запросы по id, фиксация заголовков и кодов ответа.
- Postman / Insomnia - удобное управление коллекциями запросов, переменными окружения, сохранение истории и результатов (подходит для тест-матрицы).
- Burp Suite (Proxy/Repeater/Intruder) - перехват и модификация запросов, повторные проверки, вариации параметров (edge), сравнение ответов.
- Tulip / мониторинг трафика (если доступен) - наблюдение сетевых потоков, подтверждение факта запросов чекера/атак, корреляция по времени.
- Docker (docker ps, docker logs, docker exec) - контроль состояния контейнеров, просмотр логов, перезапуск сервиса, фиксация конфигурации стенда.
- Логи приложения и БД (stdout контейнера, access/error logs, DB logs) - подтверждение прохождения сценариев и причин отказов/ошибок.
- (опционально) tcpdump/Wireshark - захват пакетов для спорных случаев (например, нестабильность, редиректы, проблемы cookie).

Артефакты (что собирать и прикладывать)

Чтобы результаты можно было проверить и повторить, для каждого сценария фиксируются:

1. HTTP-запрос (Request)
 - метод, URL, параметры (id, latest и т.п.),
 - заголовки (в т.ч. Cookie, Content-Type),
 - тело запроса (form/json),
 - время/контекст (какой пользователь, какая роль/атрибут).
2. HTTP-ответ (Response)
 - код ответа (200/401/403/404/500...),

- заголовки (Set-Cookie, Content-Type),
 - тело ответа (JSON/HTML),
 - признак утечки: наличие чужих данных/флага.
3. Снимки экрана / загрузки
- скриншоты из браузера/инструментов (Postman/Burp) для ключевых случаев,
 - сохранённые raw-дампы запросов/ответов (файлы).
4. Логи и диагностические данные
- docker logs <container> (фрагменты с временем теста),
 - сообщения об ошибках/отказах в доступе,
 - при необходимости - подтверждение из БД/сервиса, что объект создан/не создан.

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 Условия соревнования и стенд эксперимента

Соревнование проводилось в формате Attack-Defense CTF в изолированной игровой сети. Каждой команде был выделен vulnbox с фиксированным адресом вида 172.25.x.2, где x - номер команды. В рамках работы использовался vulnbox команды №2: 172.25.2.2. Взаимодействие команд происходило по сети VPN/игровой сети, а проверка сервисов выполнялась чекером организаторов по раундам.

Сдача флагов осуществлялась на сервер жюри HTTP-запросом с командным токеном, например:

- PUT http://172.24.0.1/flags с заголовком X-Team-Token и массивом флагов в теле запроса.

Также для части сервисов мог использоваться механизм attack_data (эндпоинт организаторов), предназначенный для передачи данных атаки в соответствии с правилами конкретного сервиса.

Стенд (vulnbox + Docker-контейнеры)

Исследуемые сервисы были развернуты в контейнерной среде Docker на vulnbox.

По состоянию стенда использовались следующие компоненты:

- otkritki-backend - HTTP API сервиса (порт 8083/tcp);
- otkritki-frontend - веб-интерфейс (порт 31338/tcp);
- otkritki-db (MySQL) - база данных (внутренний порт 3306/tcp, доступ из backend по сети Docker);
- zapiski - отдельный сетевой сервис (порт 5712/tcp);
- zapiski-cleaner - вспомогательный контейнер (служебные задачи, без внешнего порта).

Схема взаимодействия

Логическая схема работы стенда:

1. Пользователь/чекер обращается к frontend (:31338) и/или напрямую к backend API (:8083).
2. Backend выполняет аутентификацию (cookie-сессии), проверку прав доступа и операции с данными.
3. Данные пользователей и приватные объекты (в т.ч. содержащие флаги) хранятся в MySQL, доступ к которой осуществляется только со стороны backend.

4. Чекер в раундах выполняет типовые сценарии (регистрация/логин/создание/чтение), размещая и затем запрашивая приватные данные.

Сбор логов и наблюдение трафика (артефакты эксперимента)

Для фиксации результатов и подтверждения сценариев использовались:

- Логи контейнеров Docker: `docker logs <container>` (backend/frontend/сервисы) - для подтверждения обработки запросов и ошибок контроля доступа.
- Состояние и конфигурация стенда: `docker ps` (имена контейнеров, проброс портов).
- HTTP-артефакты тестов: сохранённые запросы/ответы (curl/Burp/Postman), включая заголовки Cookie/Set-Cookie, коды ответов и тела JSON/HTML.
- Наблюдение сетевого взаимодействия: при необходимости - анализ трафика средствами соревнования (например, Tulip/аналог) либо локальный захват трафика на vulnbox (tcpdump) для спорных случаев.

2.2 Краткое описание сервиса и модели данных

В рамках практической части рассматривался набор сервисов команды, ключевыми из которых являются Otkritki (веб-сервис «открытки») и Zapiski (сервис «заметки»). Оба сервиса реализуют типовую для A/D-формата бизнес-логику: пользователь создаёт приватные объекты, а другие пользователи не должны иметь к ним доступа. В качестве приватных данных чекер размещает флаги.

Назначение и основной функционал (Otkritki)

Сервис Otkritki предоставляет функции:

- регистрация пользователя и вход в систему;
- создание «открытки» (сообщение с текстом и параметром типа изображения);
- просмотр списка доступных пользователю открыток;
- просмотр конкретной открытки по идентификатору.

Аутентификация реализована через cookie-сессию (cookie session), внутри которой хранятся признаки authenticated, id пользователя и gender.

Сущности и модель данных (Otkritki)

В базе данных (MySQL) используются основные сущности:

1. User (пользователь)

- ID - идентификатор;
- Username - имя пользователя;
- Password - пароль (как поле модели);
- Gender - атрибут пользователя (используется в логике доступа).

2. GiftCard (открытка)

- ID - идентификатор открытки;
- To - получатель (строковое поле);
- From - отправитель (username автора);
- Text - текст открытки (здесь на практике может размещаться флаг);
- ImageType - параметр отображения/типа изображения.

Логика доступа к открыткам строится на принципе ownership: открытка доступна отправителю и получателю. Для создания открыток дополнительно применяется ограничение по атрибуту (например, «доступ только для male»).

Zapiski (общее описание)

Сервис Zapiski концептуально аналогичен по смыслу: пользователь создаёт и хранит заметки/записи, которые должны быть приватными. Приватные данные (флаги) в таком сервисе обычно размещаются в тексте заметки или связанных полях и затем извлекаются при чтении заметки владельцем.

Типовой поток чекера (размещение/чтение флага)

Проверка чекером в A/D обычно соответствует «пользовательскому сценарию» и включает:

1. Регистрация нового пользователя (случайные учётные данные).
2. Аутентификация (получение cookie-сессии).
3. Создание объекта, содержащего флаг:
 - в Otkritki - создание открытки, где флаг кладётся в поле Text;
 - в Zapiski - создание заметки, где флаг кладётся в содержимое.
4. Проверка сохранности: получение списка «своих» объектов и/или получение объекта по id.
5. Повторная проверка в следующих раундах: чекер пытается снова получить ранее сохранённый флаг. Если флаг недоступен владельцу или «исчез», это трактуется как нарушение приватности/целостности данных и может приводить к статусам CORRUPT/MUMBLE (в зависимости от характера сбоя).

Эта модель напрямую связывает корректность авторизации на уровне объектов (кто может читать конкретную открытку/заметку) с сохранностью флагов и итоговыми показателями работоспособности сервиса в соревновании.

2.3 Поверхность атаки и точки контроля доступа (Otkritki/Zapiski)

В рамках стенда Attack-Defense основная поверхность атаки формируется сетевыми сервисами, доступными другим командам, и API-методами, через которые осуществляется работа с приватными данными (в т.ч. флагами).

Сетевые компоненты и точки входа

- Otkritki (frontend): веб-интерфейс на порту 31338/tcp (HTTP).
- Otkritki (backend API): HTTP API на порту 8083/tcp.
- Zapiski: отдельный сетевой сервис на порту 5712/tcp.
- MySQL: внутренняя БД (порт 3306 внутри Docker-сети; напрямую извне обычно не экспонируется).

Таким образом, внешняя поверхность атаки для Otkritki - это прежде всего HTTP-эндпоинты backend'а и логика обработки сессий/прав доступа.

Карта основных эндпоинтов и операций (Otkritki)

(пути приведены по фактически используемым вызовам и обработчикам сервиса)

- Регистрация: POST /api/register
Создание пользователя и выдача cookie-сессии.
- Вход: POST /api/login
Аутентификация и выдача/обновление cookie-сессии.
- Получение списка объектов: GET /api/cards
Получение списка “доступных” открыток для пользователя (возможен параметр latest).
- Получение объекта по id: GET /api/card?id=<N>
Чтение конкретной открытки по идентификатору.
- Создание объекта: (обработчик AddCardPost, типично POST /api/card или POST /api/add_card)
Создание открытки (поля: to, text, imageType).
- Выход: (обработчик LogoutPost)
Завершение сессии (удаление cookie).
- Проверка авторизации: (обработчик CheckAuth)
Технический запрос “пользователь авторизован”.

Критичные параметры, влияющие на контроль доступа:

- id (в GET /api/card) - идентификатор объекта;
- latest (в GET /api/cards) - вариант выборки списка.

Cookie / Session и аутентификация

Аутентификация реализована через cookie-сессию (gorilla/sessions):

- имя cookie: session;
- в сессию сохраняются ключевые значения:
 - authenticated (bool),
 - id (uint, идентификатор пользователя),
 - gender (string).

Любые операции, связанные с чтением/созданием объектов, завязаны на наличие корректной сессии и значения id пользователя.

Middleware и точки контроля доступа

В backend выделены три ключевых middleware-уровня, которые формируют “контур” контроля доступа:

1. CorsMiddleware

Устанавливает CORS-заголовки и разрешает credentials (cookie) для фронтенда. Это важно, потому что доступ к API в браузере выполняется именно через cookie-сессию.

2. AuthMiddleware (точка AuthN/AuthZ-допуска к API)

Проверяет наличие cookie session и признак authenticated, а также валидность id пользователя через обращение к БД.

-> Это базовая “дверь” ко всем защищённым эндпоинтам.

3. MaleMiddleware (ограничение по атрибуту)

Допускает выполнение операции только при gender == male.

-> Это пример авторизации по атрибуту (частный случай ABAC).

Проверки прав на уровне объектов (ownership/roles)

Ключевая точка объектной авторизации находится в обработчике чтения открытки по id:

1. CardPage (GET /api/card?id=...)

После получения userID из сессии сервис загружает пользователя и открытку, затем выполняет проверку, что открытка “принадлежит” пользователю: доступ разрешён отправителю или получателю (ownership-логика: From/To).

2. GetCards (GET /api/cards)

Список открыток формируется на основании имени пользователя (как получателя) и параметра latest.

-> Здесь контроль доступа реализован на уровне выборки данных (пользователь получает только релевантные записи).

3. AddCardPost (создание открытки)

Доступ к созданию зависит от:

- наличия авторизации (cookie-сессия),
- прохождения ограничения MaleMiddleware,
- корректного определения отправителя по id из сессии.

2.4 Анализ логических ошибок контроля доступа

В ходе анализа сервиса (Otkritki) выявлены типовые логические дефекты контроля доступа, влияющие на приватность данных (флагов) и показатели SLA.

Ошибка 1 - обход аутентификации при входе (AuthN bypass)

- Где: обработчик входа (POST /api/login).
- Почему: проверка выполняется только по наличию пользователя, без верификации пароля.
- Как проявляется: вход возможен при любом значении пароля для существующего имени пользователя.
- Риск для флагов: злоумышленник получает сессию пользователя и доступ к его приватным объектам (потенциальная утечка флагов).
- Влияние на SLA: напрямую SLA не снижает (сервис “работает”), но повышает риск массовых утечек и статуса CORRUPT у команды.

Ошибка 2 - статический/предсказуемый ключ подписи cookie-сессии (подмена сессии)

- Где: инициализация cookie-хранилища с фиксированным ключом (CookieStore).
- Почему: ключ подписи задан константой, что повышает риск подделки cookie.
- Как проявляется: при известном ключе возможно сформировать cookie с произвольными значениями (authenticated/id/gender).
- Риск для флагов: захват чужой сессии -> доступ к чужим открыткам/данным -> утечка флагов.
- Влияние на SLA: возможны ошибки/паники при некорректных значениях сессии (риски MUMBLE/DOWN), но основное - утечки (CORRUPT).

Ошибка 3 - “substring-check” в проверке владения объектом (ошибка авторизации уровня объекта)

- Где: чтение открытки по id (GET /api/card?id=...).

- Почему: проверка получателя выполнена через поиск подстроки (нестрогое соответствие).
 - Как проявляется: пользователь с “похожим” именем может ошибочно пройти проверку доступа.
 - Риск для флагов: чтение чужих открыток (в т.ч. содержащих флаг) -> утечка.
 - Влияние на SLA: SLA обычно не падает, но возрастает вероятность CORRUPT.
- Ошибка 4 - небезопасная работа с данными сессии (DoS/нестабильность)
- Где: middleware/handlers используют значения сессии через прямое приведение типов без проверок.
 - Почему: при отсутствии ключей или неожиданных типах возможны runtime-ошибки.
 - Как проявляется: при “ломаной” сессии сервис может выдавать 5xx/падать на обработке запросов.
 - Риск для флагов: косвенный (через отказ/коррупцию доступности данных владельцу).
 - Влияние на SLA: повышение вероятности MUMBLE/DOWN (ошибки 5xx, нестабильность).

2.5 Разработка сценариев тестирования и тест-матрицы

```
cd ~/Desktop/service/checker
```

```
python3 -m venv venv
```

```
source venv/bin/activate
```

```
pip install -U pip
```

```
pip install -r requirement.txt
```

```
export PORT=10001
```

```
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$
export PORT=10001
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ mkdir -p ~/Desktop/service/evidence
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ cd ~/Desktop/service/checker
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ source venv/bin/activate
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ export PORT=10001
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py check 127.0.0.1 | tee ../evidence/check.txt
Exit with code 101
uXLNvgkquveBngpN
rLLCRLaFloniYDRE
PThHXsorsZykJO6F
HLHwYJrLBhXmGRCC
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ AUTH=$(python checker.py put 127.0.0.1 1 'FLAG{demo_flag_123}' \
| grep -v 'Exit with code' | head -n 1)
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ echo "AUTH=[$AUTH]" | tee ../evidence/auth.txt
AUTH=[XcEmphBTvYijzdlj:QmAHWhZPjBARiFbS]
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py get 127.0.0.1 "$AUTH" 'FLAG{demo_flag_123}' | tee ../evidence/get.txt
Exit with code 101
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$
```

Рисунок 5 — Сбор артефактов валидации по чекеру (CHECK/PUT/GET)

```

cat > ~/Desktop/service/checker/ac_suite.py <<'EOF'
#!/usr/bin/env python3
import os, sys, string, random
from pwn import remote

PORT = int(os.environ.get("PORT", "10001"))

def rnd(n=10):
    import random, string
    return "".join(random.choice(string.ascii_letters) for _ in range(n))

def sync_menu(io):
    io.recvuntil(b"Choose an option: ")

def press_enter(io):
    io.sendline(b"")

def register(io, u, p):
    sync_menu(io); io.sendline(b"1")
    io.sendlineafter(b"Enter new username: ", u.encode())
    io.sendlineafter(b"Enter new password: ", p.encode())
    press_enter(io)

def login(io, u, p):
    sync_menu(io); io.sendline(b"2")
    io.sendlineafter(b"Username: ", u.encode())
    io.sendlineafter(b>Password: ", p.encode())
    press_enter(io)

def add_note(io, name, content):
    sync_menu(io); io.sendline(b"3")
    io.sendlineafter(b"Enter TODO name (e.g., todo1): ", name.encode())
    io.sendlineafter(b"Enter your note content: ", content.encode())
    press_enter(io)

```

```
def list_notes(io) -> bytes:
    sync_menu(io); io.sendline(b"4")
    data = io.recvuntil(b"Press Enter to continue...", timeout=3)
    press_enter(io)
    return data
```

```
def tc_no_login_list(host):
    io = remote(host, PORT)
    out = list_notes(io)
    io.close()
    return b"You must be logged in" in out
```

```
def tc_no_login_add(host):
    io = remote(host, PORT)
    sync_menu(io); io.sendline(b"3")
    out = io.recvuntil(b"Press Enter to continue...", timeout=3)
    press_enter(io)
    io.close()
    return b"You must be logged in" in out
```

```
def tc_isolation_normal(host):
    flag = b"FLAG{isolation_test}"
    ua, pa = rnd(), rnd()
    io = remote(host, PORT)
    register(io, ua, pa); login(io, ua, pa)
    add_note(io, rnd(), flag.decode())
    io.close()
```

```
ub, pb = rnd(), rnd()
io = remote(host, PORT)
register(io, ub, pb); login(io, ub, pb)
out = list_notes(io)
io.close()
```

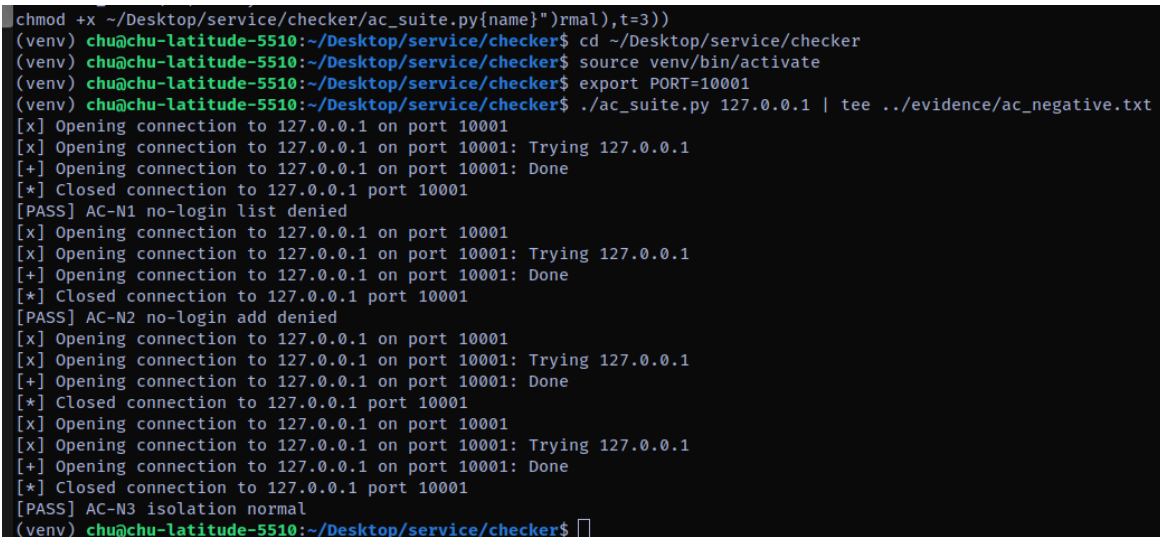
```

return flag not in out

if __name__ == "__main__":
    host = sys.argv[1] if len(sys.argv) > 1 else "127.0.0.1"
    tests = [
        ("AC-N1 no-login list denied", tc_no_login_list),
        ("AC-N2 no-login add denied", tc_no_login_add),
        ("AC-N3 isolation normal", tc_isolation_normal),
    ]
    ok = True
    for name, fn in tests:
        res = fn(host)
        print(f"[{'PASS' if res else 'FAIL'}] {name}")
        ok &= res
    sys.exit(0 if ok else 1)
EOF

```

chmod +x ~/Desktop/service/checker/ac_suite.py



```

chmod +x ~/Desktop/service/checker/ac_suite.py{name}")rmal),t=3))
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ cd ~/Desktop/service/checker
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ source venv/bin/activate
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ export PORT=10001
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$ ./ac_suite.py 127.0.0.1 | tee ../evidence/ac_negative.txt
[x] Opening connection to 127.0.0.1 on port 10001
[x] Opening connection to 127.0.0.1 on port 10001: Trying 127.0.0.1
[+] Opening connection to 127.0.0.1 on port 10001: Done
[*] Closed connection to 127.0.0.1 port 10001
[PASS] AC-N1 no-login list denied
[x] Opening connection to 127.0.0.1 on port 10001
[x] Opening connection to 127.0.0.1 on port 10001: Trying 127.0.0.1
[+] Opening connection to 127.0.0.1 on port 10001: Done
[*] Closed connection to 127.0.0.1 port 10001
[PASS] AC-N2 no-login add denied
[x] Opening connection to 127.0.0.1 on port 10001
[x] Opening connection to 127.0.0.1 on port 10001: Trying 127.0.0.1
[+] Opening connection to 127.0.0.1 on port 10001: Done
[*] Closed connection to 127.0.0.1 port 10001
[x] Opening connection to 127.0.0.1 on port 10001
[x] Opening connection to 127.0.0.1 on port 10001: Trying 127.0.0.1
[+] Opening connection to 127.0.0.1 on port 10001: Done
[*] Closed connection to 127.0.0.1 port 10001
[PASS] AC-N3 isolation normal
(venv) chu@chu-latitude-5510:~/Desktop/service/checker$

```

Рисунок 6 — Результаты негативных тестов контроля доступа (AC-N1–AC-N3)

Setup exploit venv:

cd ~/Desktop/service/sploit

python3 -m venv venv


```
pip install -r requirement.txt
```

Рисунок 7 — Edge-тест AC-E1 до исправления: запуск sploit.py подтверждает утечку флага (FLAG{...} присутствует в выводе), результат FAIL (leak).

Рисунок 8 - Уязвимый фрагмент кода в service.sh

```

chu@chu-latitude-5510: ~/Desktop/service/service
GNU nano 7.2 /home/chu/Desktop/service/service/app/service.sh
echo "[+] Login successful! Logged in as '$LOGGED_IN_USER'."
else
echo "[!] Invalid credentials!"
fi

echo
}

function add_todo() {
echo

if [ -z "$LOGGED_IN_USER" ]; then
echo "[!] You must be logged in to add notes."
return
fi

echo "[+] Add a TODO note for '$LOGGED_IN_USER' [+]"

echo -n "Enter TODO name (e.g., todo1): "
read note_name

note_path="notes/${LOGGED_IN_USER}---${note_name}"

mkdir -p "notes"

echo -n "Enter your note content: "
read note_content

printf '%s\n' "$note_content" > "$note_path"

echo "[+] Note saved to '$note_path'."
echo
}

[ Wrote 158 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_/ Go To Line  M-E Redo      M-6 Copy

```

Рисунок 9 — Исправленный фрагмент кода в service.sh

```

=> resolving provenance for metadata file 0.0s
[+] build 1/1
✓ Image service-todoapp Built 12.1s
(chu@chu-latitude-5510:~/Desktop/service/service$ docker compose up -d --force-recreate
[+] up 2/2
✓ Network service_default Created 0.0s
✓ Container service-todoapp-1 Created 0.1s
(chu@chu-latitude-5510:~/Desktop/service/service$ docker exec -it service-todoapp-1 bash -lc "grep -n 'bash -c' /app/service.sh
|| echo 'OK: fixed'
OK: fixed
(chu@chu-latitude-5510:~/Desktop/service/service$ cd ~/Desktop/service/checker
(chu@chu-latitude-5510:~/Desktop/service/checker$ source venv/bin/activate
(chu@chu-latitude-5510:~/Desktop/service/checker$ export PORT=10001
(chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py check 127.0.0.1
0LxhLcnpZKCuIDzK
BRKwWwotWuJozgKV
cufMyqJtWxByazhr
YcejtiHctQaxiQCo
Exit with code 101
(chu@chu-latitude-5510:~/Desktop/service/checker$ AUTH=$(python checker.py put 127.0.0.1 1 'FLAG{demo_flag_123}' | grep -v 'Exit
with code' | head -n 1)
Exit with code 101
(chu@chu-latitude-5510:~/Desktop/service/checker$ python checker.py get 127.0.0.1 "$AUTH" 'FLAG{demo_flag_123}'
Exit with code 101
(chu@chu-latitude-5510:~/Desktop/service/checker$ cd ~/Desktop/service/sploit
(chu@chu-latitude-5510:~/Desktop/service/sploit$ source venv/bin/activate
(chu@chu-latitude-5510:~/Desktop/service/sploit$ python exploit.py 127.0.0.1 10001 | tee ../evidence/sploit_after_fix.txt
[X] Opening connection to 127.0.0.1 on port 10001
[X] Opening connection to 127.0.0.1 on port 10001: Trying 127.0.0.1
[+] Opening connection to 127.0.0.1 on port 10001: Done
[X] Receiving all data
[X] Receiving all data: 0B
[X] Receiving all data: 1B
[X] Receiving all data: 179B
[+] Receiving all data: Done (179B)
[*] Closed connection to 127.0.0.1 port 10001
b"\n[+] TODO notes for 'ZXTSypZYrUMngJCM' [+]\n-----\nTODO: TkdiJotVdBgunKk\n$(cat ./notes/*)\n-----
-----\n\nPress Enter to continue...\n"
(chu@chu-latitude-5510:~/Desktop/service/sploit$ grep -n "FLAG{" ../evidence/sploit_after_fix.txt && echo "FAIL (still leaks)" |
| echo "PASS (fixed)"
PASS (fixed)
(chu@chu-latitude-5510:~/Desktop/service/sploit$

```

Рисунок 10 — Валидация после исправления: контейнер пересоздан, чекер (CHECK/PUT/GET) возвращает OK (код 101), а повторный запуск exploit.py не выявляет FLAG{...} (PASS, утечки нет).

2.6 Рекомендации и валидация

Рекомендации по повышению защищённости

1. Исключить интерпретацию пользовательского ввода оболочкой (ключевая мера).

Выявленная эксплуатация была основана на выполнении введённого пользователем содержимого как команды shell, что приводило к чтению notes/* и утечке флагов. Для устранения причины уязвимости необходимо полностью отказаться от конструкций вида `bash -c`, `eval`, подстановок `$()/backticks` при обработке пользовательского ввода. Запись данных должна выполняться безопасно, без запуска оболочки (например, через `printf '%s\n' "$note_content" > "$note_path"`). Это предотвращает выполнение вредоносных конструкций и блокирует утечки.

2. Ввести allowlist-валидацию входных параметров, влияющих на идентификаторы объектов.

Поля `username`, `note_name` фактически формируют идентификатор/путь объекта (файла заметки), поэтому их необходимо ограничивать безопасным набором символов и длиной (например, `[A-Za-z0-9_-]{1,32}`), запрещая `../`, пробелы, wildcard-символы и управляющие последовательности. Мера снижает риск обходов изоляции (`path traversal/glob/word-splitting`) и стабилизирует поведение сервиса под нагрузкой.

3. Жёсткая object-level изоляция данных.

Объекты (заметки) должны быть однозначно привязаны к владельцу, а операции чтения/перечисления — работать только в рамках пространства владельца. Практически это означает: формирование имени файла по детерминированному шаблону, корректное экранирование/квотирование переменных, безопасный перебор файлов (без раскрытия чужих объектов при нештатных именах).

4. Безопасная обработка ошибок без утечек.

Сообщения об ошибках должны быть информативными для пользователя, но не раскрывать внутренние пути, структуру хранилища или данные других пользователей. В условиях Attack-Defense это снижает качество “подсказок” атакующему и уменьшает скорость разработки эксплойта.

5. Регрессионная проверка после патчей как обязательное правило A/D.

Любые изменения должны сопровождаться обязательным прогоном чекера и тест-матрицы, чтобы не допустить перехода сервиса в состояния

MUMBLE/CORRUPT/DOWN и не ухудшить SLA.

Валидация исправлений (процедура “до/после”)

Валидация выполнялась в двух частях: (1) проверка работоспособности по сценарию чекера и (2) проверка устранения утечки по edge-тесту.

1. Регрессия чекером (CHECK/PUT/GET).

После пересборки контейнера и принудительного пересоздания (`docker compose up -d --force-recreate`) сервис был проверен штатным чекером:

- `checker.py check` завершился со статусом OK (код 101), что подтверждает доступность и корректность базового сценария.
- `checker.py put` успешно разместил тестовый флаг и вернул `auth_data`.
- `checker.py get` по полученному `auth_data` извлёк ранее записанный флаг со статусом OK (код 101).

Таким образом, исправления не нарушили легитимный функционал и не привели к MUMBLE/CORRUPT.

2. Повтор edge-теста (AC-E1) на утечку через внедрение команд.

После исправления была повторно выполнена эксплуатационная проверка (`sploit.py`) с полезной нагрузкой `$(cat ./notes/*)`.

- Критерий FAIL (до исправления): в выводе присутствует `FLAG{...}`, что указывает на чтение чужих заметок/флагов.
- Критерий PASS (после исправления): в файле `sploit_after_fix.txt` отсутствуют строки `FLAG{...}`, а полезная нагрузка отображается как обычный текст, не выполняясь.

Итог валидации: уязвимость, приводившая к нарушению контроля доступа на уровне объектов и компрометации флагов, устранена; при этом сервис сохраняет работоспособность и успешно проходит чекерные сценарии в формате Attack-Defense CTF.

ЗАКЛЮЧЕНИЕ

В рамках работы выполнен анализ логических ошибок контроля доступа и разработаны практические сценарии тестирования, ориентированные на условия соревнований Attack-Defense CTF. В качестве тестового стенда использовался сервис управления заметками, для которого была построена упрощённая модель контроля доступа (субъекты — пользователи, объекты — заметки/файлы, операции — создание и просмотр), а также сформирована тест-матрица, включающая регрессионные проверки по чекеру и набор негативных/граничных тестов для выявления обходов логики.

Экспериментальная часть показала, что даже при наличии базовых ограничений (запрет операций без аутентификации и корректная “нормальная” изоляция при штатном использовании), сервис оставался уязвимым к сценарию обхода контроля доступа на уровне объектов: через небезопасную обработку пользовательского ввода атакующий мог добиться чтения содержимого чужих заметок и раскрытия флагов. Данный результат подтверждает типичную для Attack-Defense особенность — ошибки обработки входных данных и логические дефекты могут приводить к прямой компрометации флагов, что критично для итогового результата команды.

Для устранения проблемы были внесены защитные изменения, исключающие выполнение пользовательского ввода оболочкой, а также выполнена валидация исправлений по методике “до/после”. После пересборки и принудительного пересоздания контейнера сервис сохранил корректную работоспособность по сценарию чекера (CHECK/PUT/GET — статус OK), а повторный запуск эксплуатационного сценария подтвердил отсутствие утечки FLAG{...}. Это демонстрирует, что предложенный подход к тестированию и валидации позволяет одновременно предотвращать компрометацию флагов и не ухудшать SLA, избегая состояний MUMBLE/CORRUPT/DOWN.

Таким образом, поставленная цель достигнута: сформированы и апробированы сценарии тестирования логических ошибок контроля доступа в A/D-среде, а также подтверждена эффективность минимальных исправлений, совместимых с требованиями соревнований. В дальнейшем целесообразно расширить тест-матрицу на дополнительные классы логических обходов (IDOR/parameter tampering/TOCTOU), автоматизировать прогон тестов по раундам и внедрить базовое журналирование событий безопасности для ускорения реакции в условиях динамичной атакующей нагрузки.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Attack-Defense CTF: описание и правила [Электронный ресурс]. — Режим доступа: <https://cbsctf.ru/ad> (дата обращения: 12.02.2026).
2. SlideServe. Capture the Flag (CTF) [Электронный ресурс]. — Режим доступа: <https://www.slideserve.com/caleb-kim/capture-the-flag-ctf> (дата обращения: 12.02.2026).
3. OWASP Cheat Sheet Series. Authentication Cheat Sheet [Электронный ресурс]. — Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html (дата обращения: 12.02.2026).
4. OWASP Cheat Sheet Series. Access Control Cheat Sheet [Электронный ресурс]. — Режим доступа: https://cheatsheetseries.owasp.org/cheatsheets/Access_Control_Cheat_Sheet.html (дата обращения: 12.02.2026).
5. OWASP Top 10 (2021). A01:2021 — Broken Access Control [Электронный ресурс]. — Режим доступа: https://owasp.org/Top10/A01_2021-Broken_Access_Control/ (дата обращения: 12.02.2026).
6. Авторизация: определение и методы [Электронный ресурс] // Ekassir. — Режим доступа: <https://ekassir.com/blog/avtorizacziya-opredelenie-i-metody/> (дата обращения: 12.02.2026).
7. Vulnerable19. IDOR (Insecure Direct Object Reference) [Электронный ресурс]. — Режим доступа: <https://medium.com/@vulnerable19/idor-insecure-direct-object-reference-cb4a54fd8a0a> (дата обращения: 12.02.2026).
8. Gorilla Sessions [Электронный ресурс]. — Режим доступа: <https://github.com/gorilla/sessions> (дата обращения: 12.02.2026).
9. Docker Documentation: Docker CLI — logs [Электронный ресурс]. — Режим доступа: <https://docs.docker.com/reference/cli/docker/container/logs/> (дата обращения: 12.02.2026).
10. OpenAttackDefenseTools (подборка инструментов A/D) [Электронный ресурс]. — Режим доступа: <https://github.com/OpenAttackDefenseTools> (дата обращения: 12.02.2026).